ANAND AMBASTHA

EXPERIMENT 1: DFT and IDFT

Aim

To calculate the Discrete Fourier Transform and Inverse Discrete Fourier Transform of given sequences.

MATLAB Code

```
clc; clear all; close all;

x = [1 3 -1 -2];
N = length(x);

for k = 0:N-1
    X(k+1) = 0;
    for n = 0:N-1
        X(k+1) = X(k+1) + x(n+1)*exp(-1i*2*pi*k*n/N);
    end
end

disp('DFT:');
disp(X);

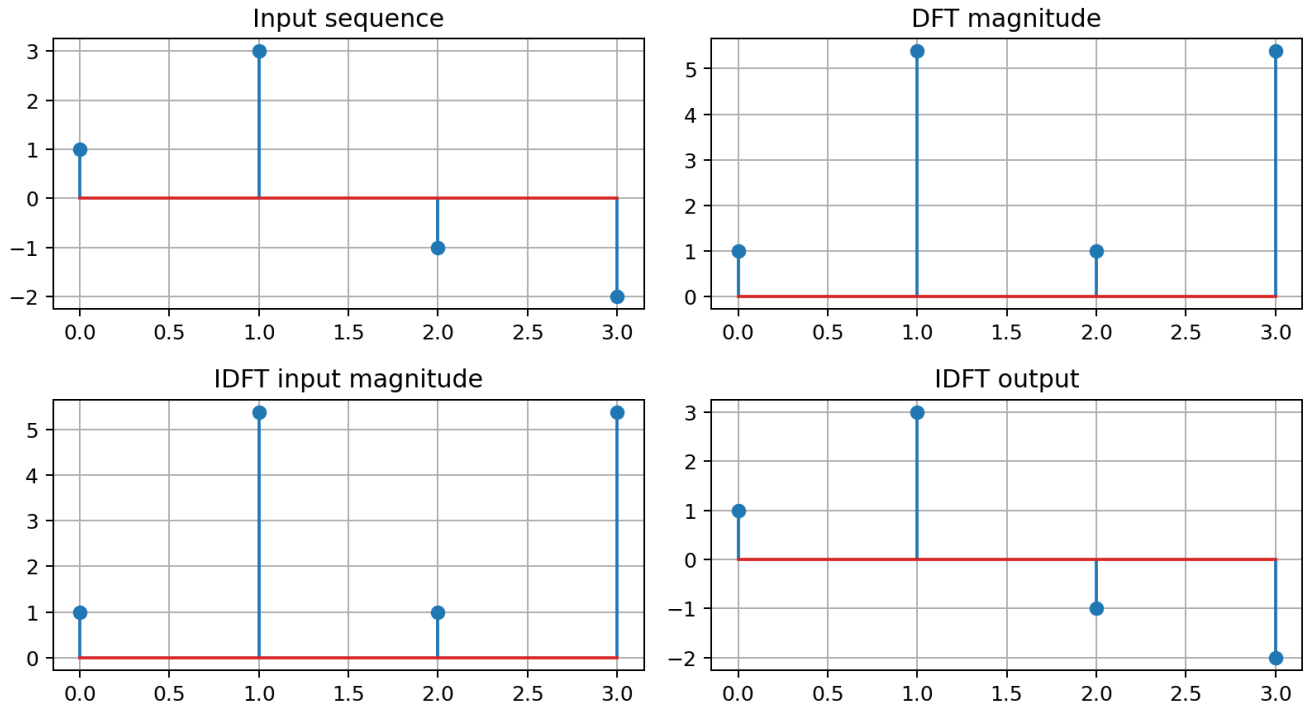
X = [1 2-5i -1 2+5i];

for n = 0:N-1
    x(n+1) = 0;
    for k = 0:N-1
        x(n+1) = x(n+1) + X(k+1)*exp(1i*2*pi*k*n/N);
    end
    x(n+1) = x(n+1)/N;
end

disp('IDFT:');
disp(x);
```

Generated Output

```
Input sequence: [ 1.+0.j  3.+0.j -1.+0.j -2.+0.j]
DFT: [ 1.+0.j  2.-5.j -1.+0.j  2.+5.j]
IDFT input: [ 1.+0.j  2.-5.j -1.+0.j  2.+5.j]
IDFT output: [ 1.+0.j  3.-0.j -1.+0.j -2.+0.j]
```



EXPERIMENT 2: Verification of Parseval's Theorem

Aim

To verify Parseval's theorem by comparing the time-domain and frequency-domain energy of a sequence.

MATLAB Code

```
clc; clear all; close all;

x = [0 2 4 6 8 10 12 14 16];
N = length(x);

X = fft(x);

LHS = sum(abs(x).^2);
RHS = sum(abs(X).^2)/N;

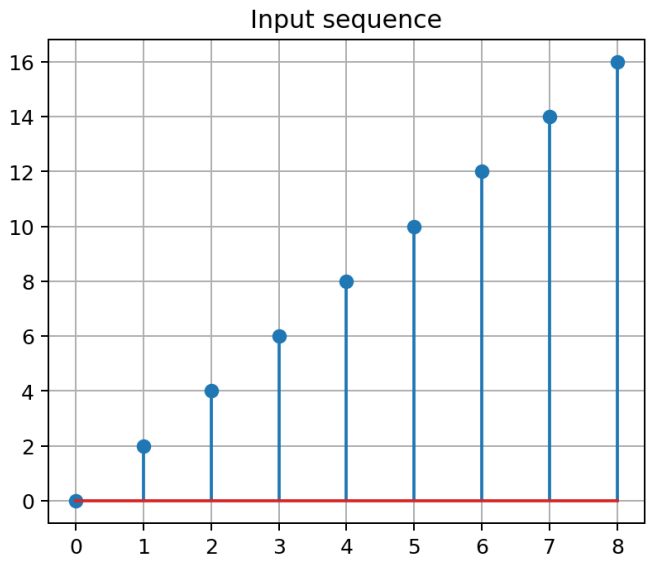
disp('Time domain energy:');
disp(LHS);

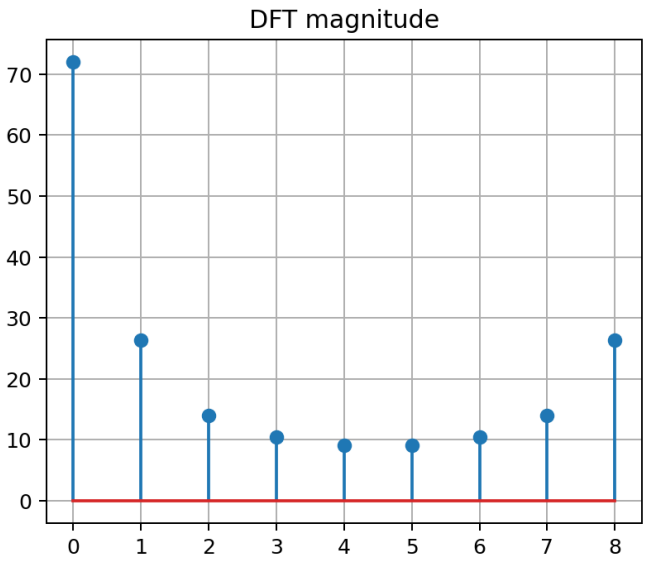
disp('Frequency domain energy:');
disp(RHS);

if abs(LHS-RHS) < 1e-10
    disp('Parseval theorem verified');
end
```

Generated Output

```
Input: [ 0  2  4  6  8 10 12 14 16]
DFT: [72. -0.j    -9.+24.727j -9.+10.726j -9. +5.196j -9. +1.587j -9. -1.587j
 -9. -5.196j -9.-10.726j -9.-24.727j]
Time-domain energy: 816.0000
Frequency-domain energy: 816.0000
Parseval verified: True
```





EXPERIMENT 3: Circular Time Shifting and Time Reversal

Aim

To study circular time shifting and time reversal of a discrete-time sequence and their effects on the DFT.

MATLAB Code

```
clc; clear all; close all;

x = [0 2 4 6 8 10 12 14 16];
N = length(x);

% Circular time shifting
y = circshift(x, [0 5]);
X = fft(y);

figure;
subplot(2,1,1);
stem(abs(X));
title('Magnitude of DFT after circular shift');

subplot(2,1,2);
stem(angle(X));
title('Phase of DFT after circular shift');

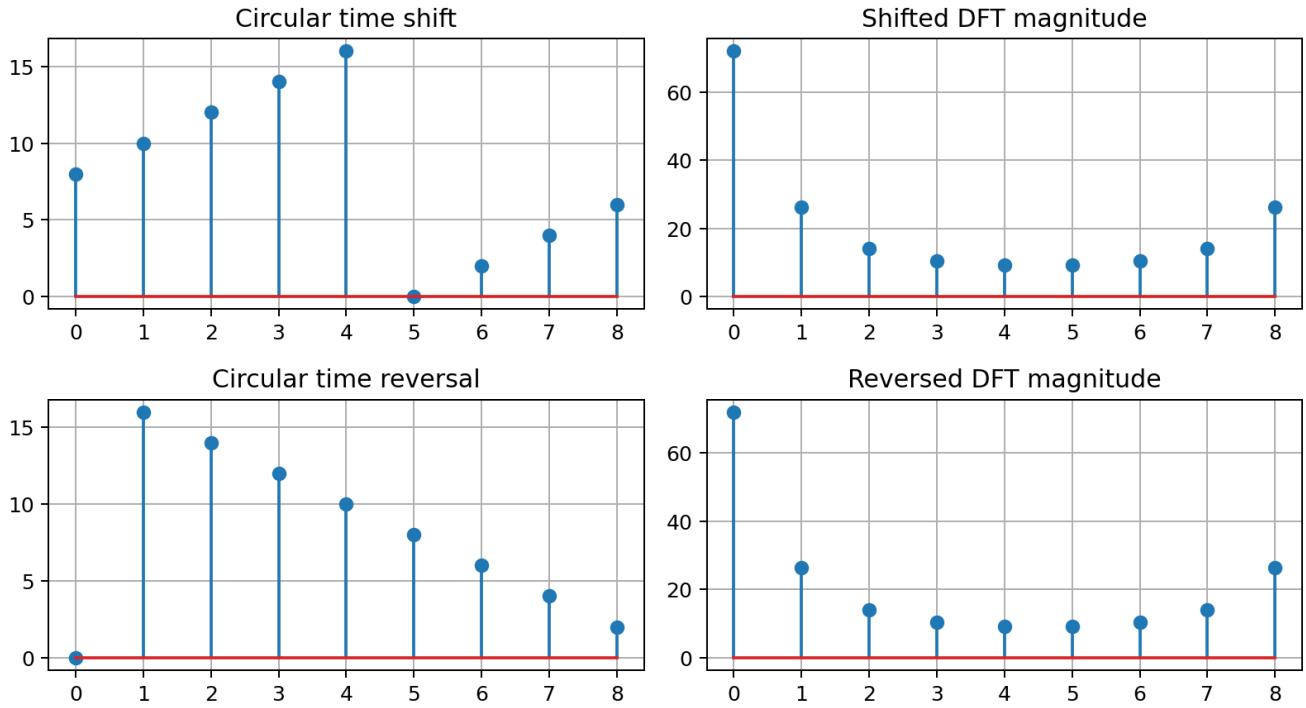
% Circular time reversal
y = x([1 N:-1:2]);
X = fft(y);

figure;
subplot(2,1,1);
stem(abs(X));
title('Magnitude of DFT after time reversal');

subplot(2,1,2);
stem(angle(X));
title('Phase of DFT after time reversal');
```

Generated Output

```
Original sequence: [ 0  2  4  6  8 10 12 14 16]
Shifted sequence: [ 8 10 12 14 16  0  2  4  6]
Reversed sequence: [ 0 16 14 12 10  8  6  4  2]
DFT of shifted sequence: [72. -0.j     0.-26.314j  0.+14.002j  0.-10.392j -0. +9.139j -0. -9.139j
  0.+10.392j  0.-14.002j  0.+26.314j]
DFT of reversed sequence: [72. -0.j    -9.-24.727j -9.-10.726j -9. -5.196j -9. -1.587j -9. +1.587j
 -9. +5.196j -9.+10.726j -9.+24.727j]
```



EXPERIMENT 4: Linear Convolution

Aim

To calculate the linear convolution of two discrete-time sequences.

MATLAB Code

```
clc; clear all; close all;

x = [1 2 0 1];
h = [1 5 7 3];

n1 = -2:1;
n2 = -1:2;

y = conv(x,h);
n = -3:3;

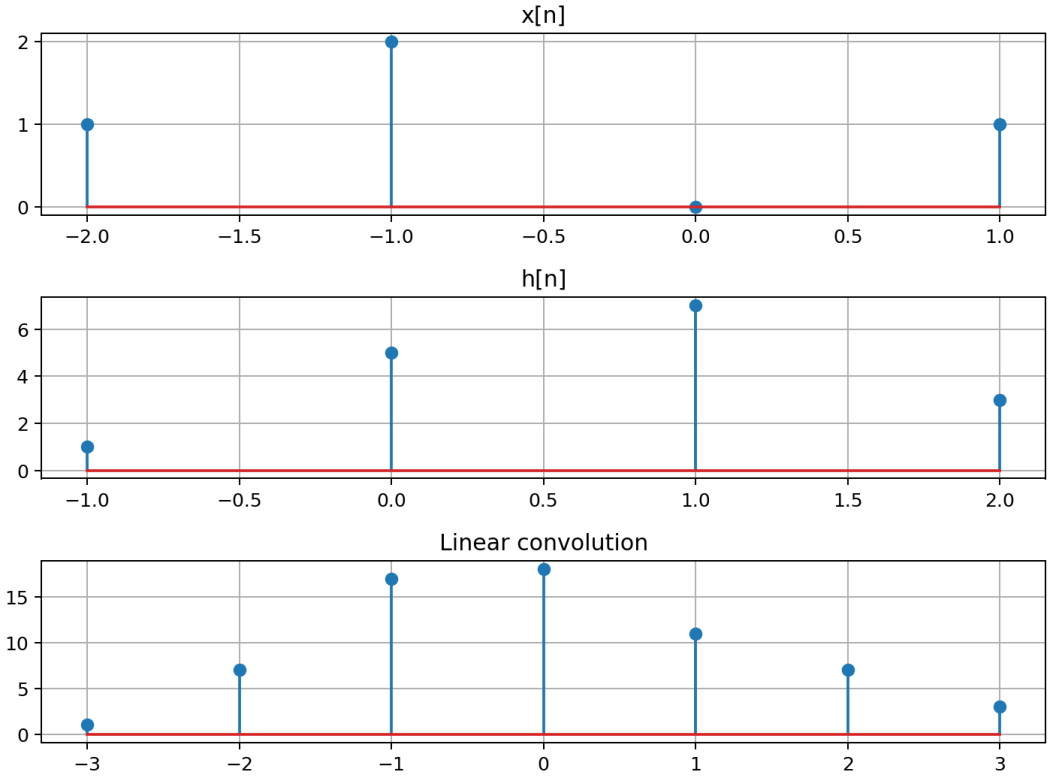
subplot(3,1,1);
stem(n1,x);
title('Input x[n]');

subplot(3,1,2);
stem(n2,h);
title('Impulse response h[n]');

subplot(3,1,3);
stem(n,y);
title('Linear convolution');
```

Generated Output

```
x[n] = [1 2 0 1]
h[n] = [1 5 7 3]
Linear convolution = [ 1  7 17 18 11  7  3]
Output index = [-3 -2 -1  0  1  2  3]
```



EXPERIMENT 5: Circular Convolution

Aim

To calculate the circular convolution of two finite-length discrete-time sequences.

MATLAB Code

```
clc; clear all; close all;

x = [1 2 0 1];
h = [1 5 7 3];

N1 = length(x);
N2 = length(h);
N = max(N1,N2);

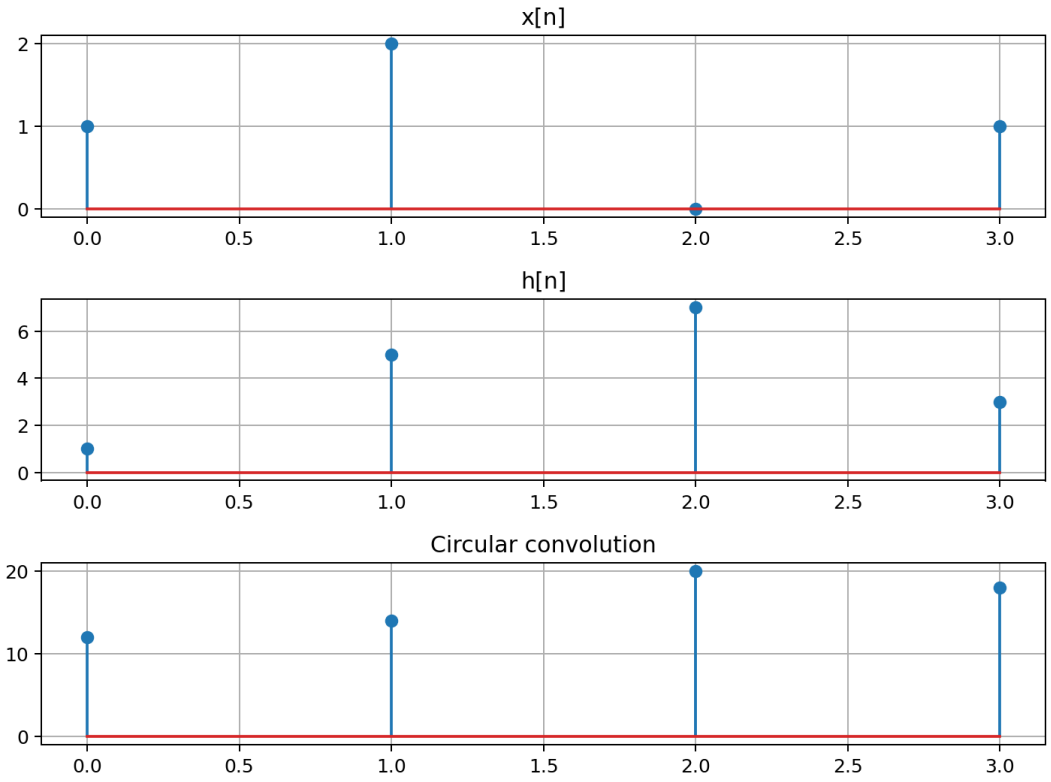
x = [x zeros(1,N-N1)];
h = [h zeros(1,N-N2)];

for n = 0:N-1
    y(n+1) = 0;
    for k = 0:N-1
        y(n+1) = y(n+1) + x(k+1)*h(mod(n-k,N)+1);
    end
end

disp('Circular convolution:');
disp(y);
stem(y);
```

Generated Output

```
x[n] = [1 2 0 1]
h[n] = [1 5 7 3]
Circular convolution length = 4
Result = [12. 14. 20. 18.]
```



EXPERIMENT 6: Linear and Circular Convolution Relationship

Aim

To obtain circular convolution by folding linear convolution and to obtain linear convolution using zero-padded circular convolution.

MATLAB Code

```
clc; clear all; close all;

x = [1 2 0 1];
h = [1 5 7 3];

% Linear convolution
y = conv(x,h);

% Circular convolution by folding
N = max(length(x),length(h));
yc = zeros(1,N);

for k = 1:length(y)
    index = mod(k-1,N)+1;
    yc(index) = yc(index)+y(k);
end

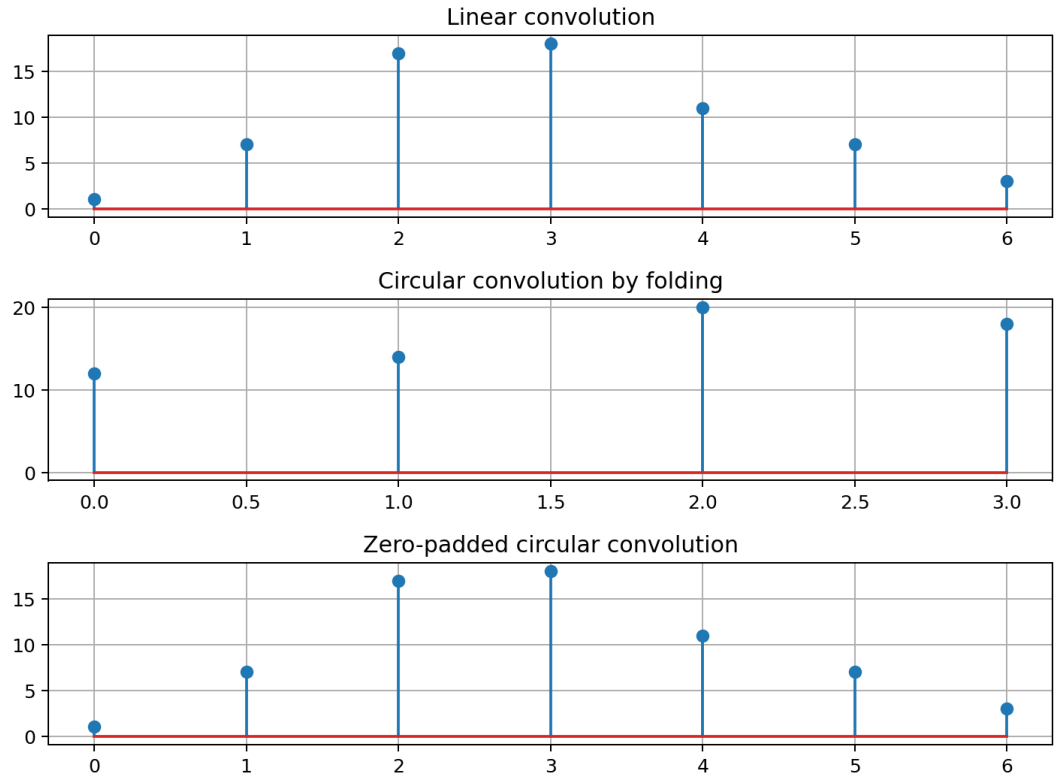
% Linear convolution using circular convolution
M = length(x)+length(h)-1;
x1 = [x zeros(1,M-length(x))];
h1 = [h zeros(1,M-length(h))];

for n = 0:M-1
    z(n+1) = 0;
    for k = 0:M-1
        z(n+1) = z(n+1)+x1(k+1)*h1(mod(n-k,M)+1);
    end
end

disp('Linear convolution:');
disp(y);
disp('Circular convolution by folding:');
disp(yc);
disp('Zero-padded circular convolution:');
disp(z);
```

Generated Output

```
Linear convolution = [ 1  7 17 18 11  7  3]
Folded circular convolution = [12. 14. 20. 18.]
Zero-padded circular convolution = [ 1.  7. 17. 18. 11.  7.  3.]
Zero-padded result equals linear convolution: True
```



EXPERIMENT 7: FIR Low-Pass Filter Using Rectangular Window

Aim

To design an FIR low-pass filter using a rectangular window and plot its frequency response.

MATLAB Code

```
clc; clear all; close all;

rp = 0.05;
rs = 0.04;
fp = 1200;
fs = 2000;
f = 9000;

num = -20*log10(sqrt(rp*rs))-13;
dem = 14.6*(fs-fp)/f;
n = ceil(num/dem);

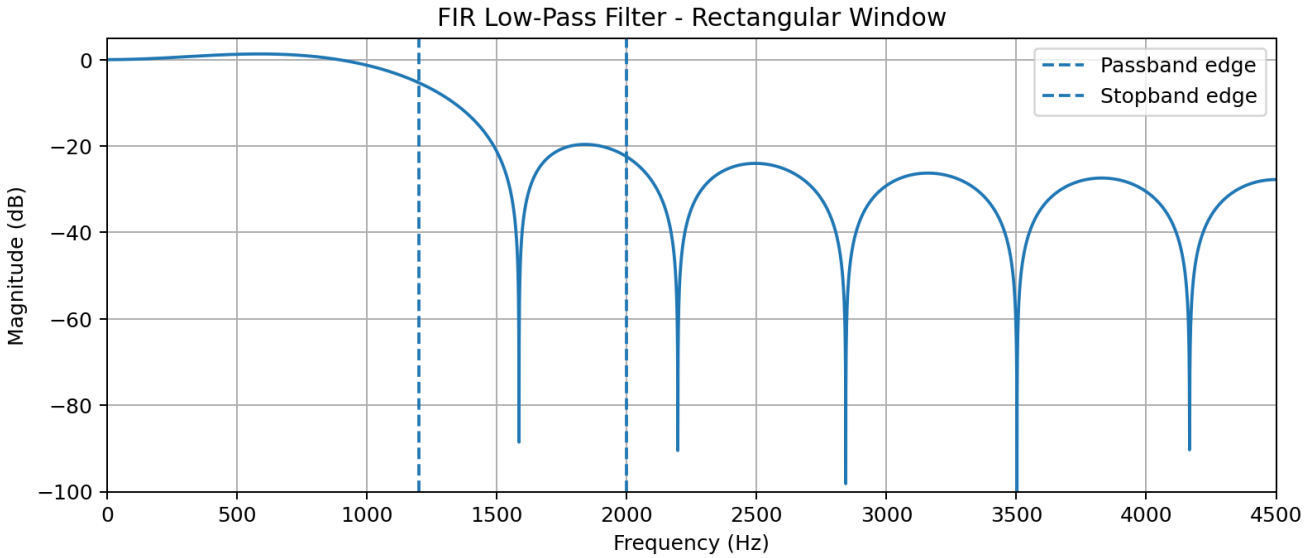
if mod(n,2) ~= 0
    n = n+1;
end

wp = 2*fp/f;
y = rectwin(n+1);
b = fir1(n,wp,y);

freqz(b,1,8192,f);
title('FIR Low Pass Filter using Rectangular Window');
```

Generated Output

```
Filter order = 12
Number of coefficients = 13
Coefficients = [-0.053859 -0.058852 -0.017661  0.066573  0.168962  0.252509  0.284657
  0.252509  0.168962  0.066573 -0.017661 -0.058852 -0.053859]
```



EXPERIMENT 8: FIR High-Pass Filter Using Rectangular Window

Aim

To design an FIR high-pass filter using a rectangular window and plot its frequency response.

MATLAB Code

```
clc; clear all; close all;

rp = 0.05;
rs = 0.04;
fp = 1500;
fs = 2000;
f = 9000;

num = -20*log10(sqrt(rp*rs))-13;
dem = 14.6*(fs-fp)/f;
n = ceil(num/dem);

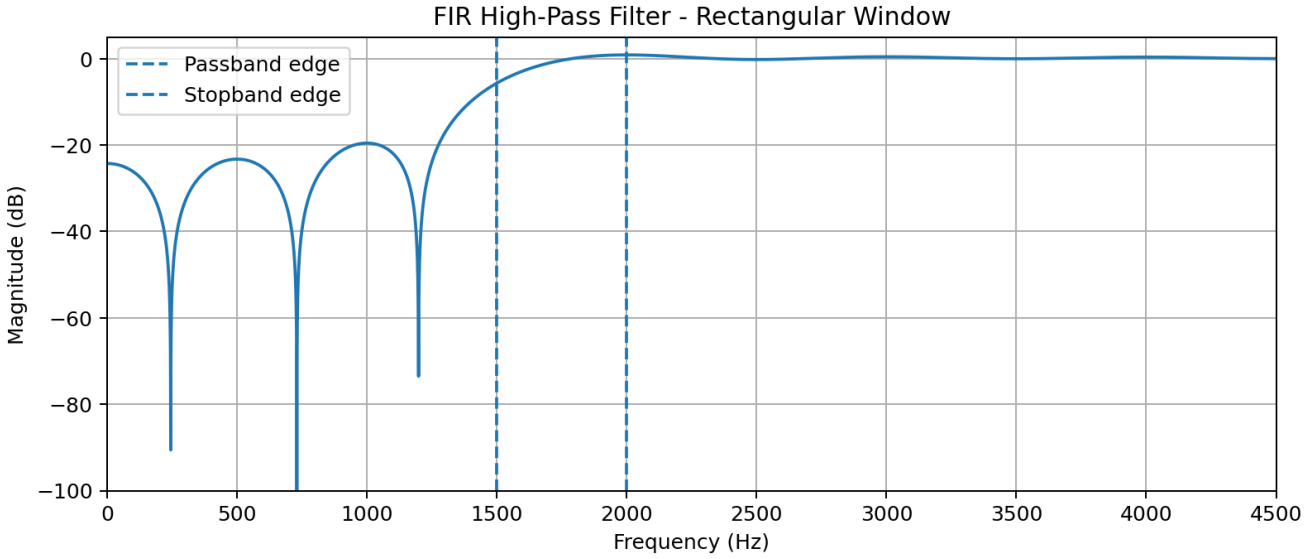
if mod(n,2) ~= 0
    n = n+1;
end

wp = 2*fp/f;
y = rectwin(n+1);
b = fir1(n,wp,'high',y);

freqz(b,1,8192,f);
title('FIR High Pass Filter using Rectangular Window');
```

Generated Output

```
Filter order = 18
Number of coefficients = 19
Coefficients = [ 0.       -0.035171 -0.040195 -0.        0.056273  0.070341  0.
 -0.140682 -0.281364  0.680451 -0.281364 -0.140682  0.        0.070341
  0.056273 -0.       -0.040195 -0.035171  0.      ]
```



EXPERIMENT 9: Butterworth Low-Pass Filter

Aim

To design a digital Butterworth low-pass filter according to the given passband and stopband specifications.

MATLAB Code

```
clc; clear all; close all;

Ap = 1;
Ast = 40;
Fpass = 100;
Fstop = 150;
Fs = 1000;

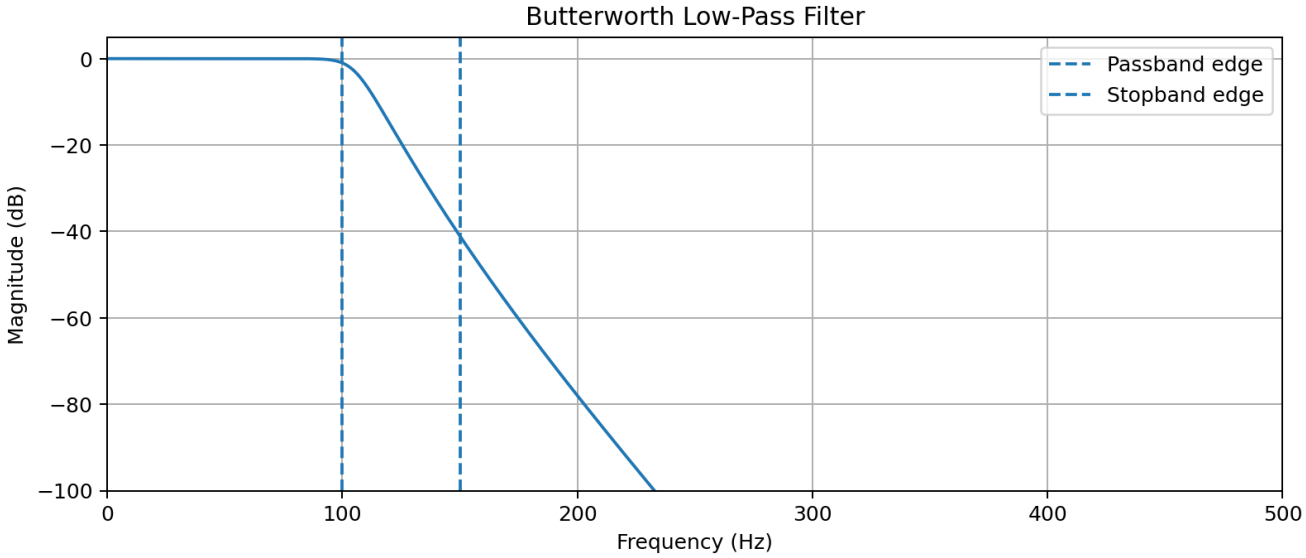
Wpass = 2*Fpass/Fs;
Wstop = 2*Fstop/Fs;

[n,Wn] = buttord(Wpass,Wstop,Ap,Ast);
[b,a] = butter(n,Wn,'low');

freqz(b,a,8192,Fs);
title('Butterworth Low Pass Filter');
```

Generated Output

```
Filter order = 12
Critical frequency = 0.210775
Numerator coefficients = [0.00e+00 2.00e-06 1.30e-05 4.40e-05 1.00e-04 1.60e-04 1.87e-04 1.60e-04
 1.00e-04 4.40e-05 1.30e-05 2.00e-06 0.00e+00]
Denominator coefficients = [ 1.0000000e+00 -6.9308420e+00  2.2718426e+01 -4.6329619e+01
  6.5230969e+01 -6.6623099e+01  5.0505746e+01 -2.8584848e+01
  1.1970505e+01 -3.6129270e+00  7.4523800e-01 -9.4242000e-02
  5.5210000e-03]
```



EXPERIMENT 10: Chebyshev Type-I Low-Pass Filter

Aim

To design a digital Chebyshev Type-I low-pass filter and plot its frequency response.

MATLAB Code

```
clc; clear all; close all;

delta1 = 1;
delta2 = 40;

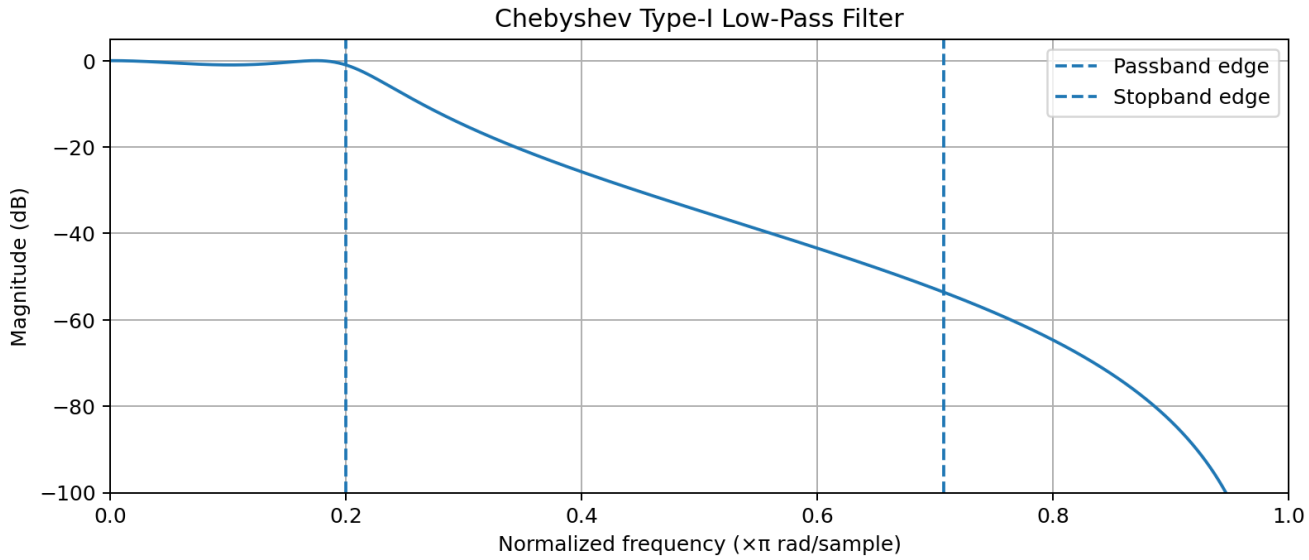
% Assumed corrected low-pass frequency ordering
wp = 0.2;
ws = 0.707;

[n,Wn] = cheb1ord(wp,ws,delta1,delta2);
[b,a] = cheby1(n,delta1,Wn,'low');

freqz(b,a,8192);
title('Chebyshev Type-I Low Pass Filter');
```

Generated Output

```
Filter order = 3
Critical frequency = 0.200000
Numerator coefficients = [0.011475 0.034424 0.034424 0.011475]
Denominator coefficients = [ 1.       -2.13779   1.769346 -0.539758]
Assumption: Wpass=0.2 and Wstop=0.707.
```



EXPERIMENT 11: Decimation

Aim

To reduce the sampling rate of a discrete-time signal by a factor of three.

MATLAB Code

```
clc; clear all; close all;

t = 0:0.01:1;
D = 3;

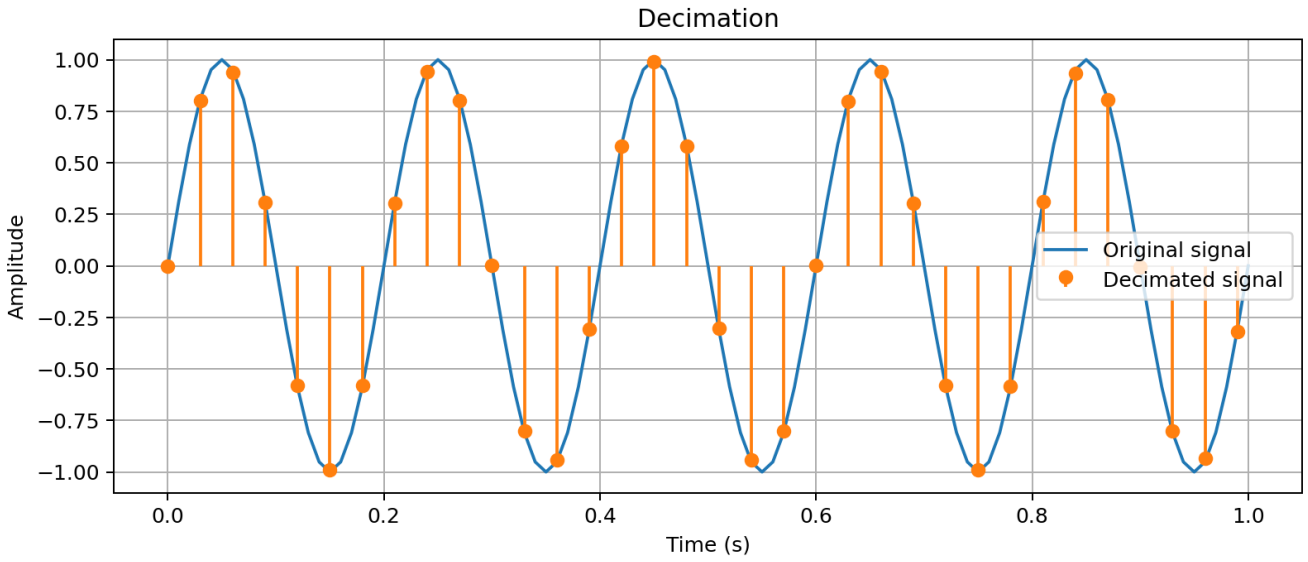
x = sin(2*pi*5*t);
y = decimate(x,D);

subplot(2,1,1);
plot(t,x);
title('Original Signal');

subplot(2,1,2);
stem(y);
title('Decimated Signal');
```

Generated Output

```
Original samples = 101
Decimation factor = 3
Decimated samples = 34
Original sampling frequency = 100 Hz
New sampling frequency = 33.3333 Hz
Decimated sequence = [-1.800e-03  8.024e-01  9.387e-01  3.071e-01 -5.820e-01 -9.893e-01
 -5.807e-01  3.047e-01  9.417e-01  7.994e-01  5.000e-04 -8.005e-01
 -9.408e-01 -3.052e-01  5.807e-01  9.901e-01  5.804e-01 -3.048e-01
 -9.411e-01 -8.005e-01  1.200e-03  7.982e-01  9.432e-01  3.034e-01
 -5.801e-01 -9.887e-01 -5.841e-01  3.105e-01  9.346e-01  8.063e-01
 -4.100e-03 -7.999e-01 -9.350e-01 -3.196e-01]
```



EXPERIMENT 12: Interpolation

Aim

To increase the sampling rate of a decimated signal using interpolation.

MATLAB Code

```
clc; clear all; close all;

t = 0:0.01:1;
d = 3;

x = sin(2*pi*5*t);

y = decimate(x,d);
y1 = interp(y,d);

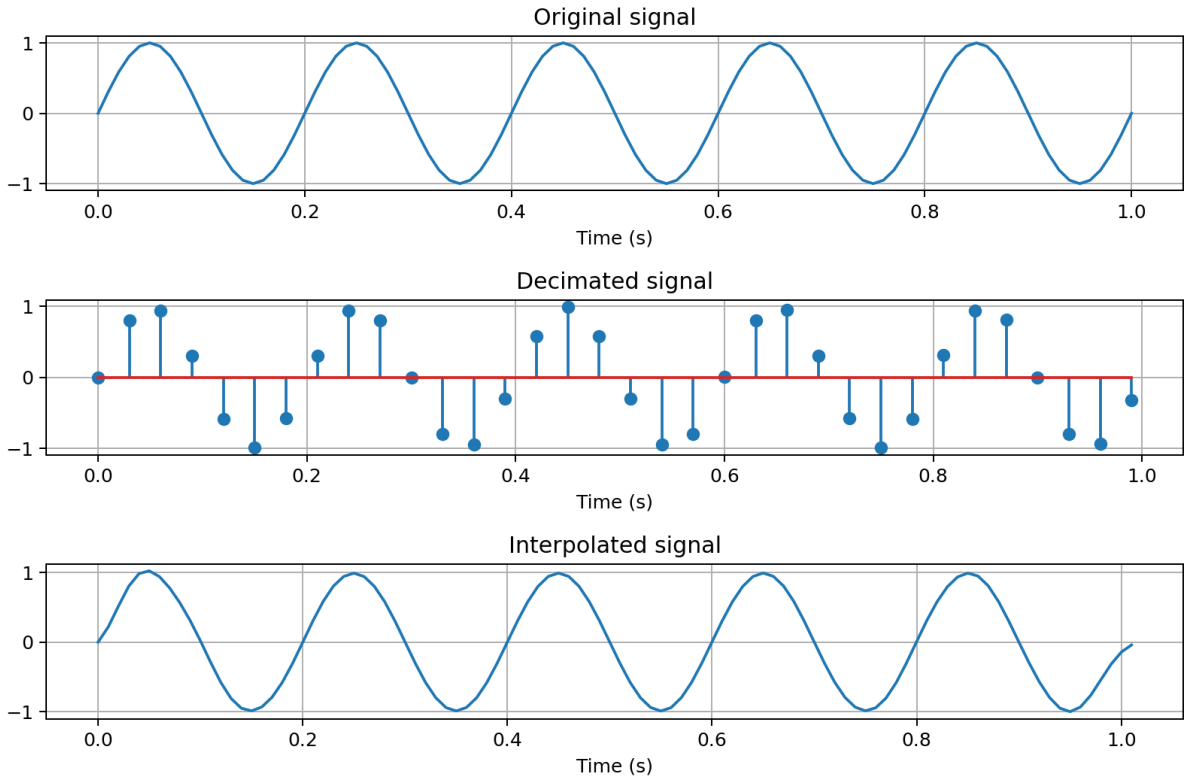
subplot(3,1,1);
plot(x);
title('Original Signal');

subplot(3,1,2);
stem(y);
title('Decimated Signal');

subplot(3,1,3);
plot(y1);
title('Interpolated Signal');
```

Generated Output

```
Original samples = 101
Decimated samples = 34
Interpolated samples = 102
Interpolation factor = 3
Interpolated sequence = [-1.8000e-03  2.1840e-01  5.1910e-01  8.0290e-01  9.8340e-01  1.0230e+00
  9.3920e-01  7.7530e-01  5.6150e-01  3.0730e-01  1.6800e-02 -2.9210e-01
 -5.8240e-01 -8.1250e-01 -9.5170e-01 -9.8990e-01 -9.3430e-01 -7.9450e-01
 -5.8100e-01 -3.0990e-01 -4.2000e-03  3.0490e-01  5.8410e-01  8.0370e-01
  9.4230e-01  9.8960e-01  9.4080e-01  7.9990e-01  5.8140e-01  3.0610e-01
  5.0000e-04 -3.0540e-01 -5.8170e-01 -8.0090e-01 -9.4300e-01 -9.9140e-01
 -9.4140e-01 -8.0090e-01 -5.8160e-01 -3.0540e-01 -2.0000e-04  3.0520e-01
  5.8100e-01  8.0150e-01  9.4280e-01  9.9070e-01  9.4270e-01  8.0120e-01
  5.8080e-01  3.0510e-01 -0.0000e+00 -3.0500e-01 -5.8110e-01 -8.0070e-01
 -9.4170e-01 -9.9210e-01 -9.4360e-01 -8.0100e-01 -5.8110e-01 -3.0440e-01
  1.2000e-03  3.0580e-01  5.8040e-01  7.9870e-01  9.4150e-01  9.9190e-01
  9.4380e-01  8.0340e-01  5.8230e-01  3.0350e-01 -3.2000e-03 -3.0760e-01
 -5.8050e-01 -7.9710e-01 -9.3760e-01 -9.8930e-01 -9.4600e-01 -8.0690e-01
 -5.8450e-01 -3.0300e-01  6.7000e-03  3.1070e-01  5.8040e-01  7.9390e-01
  9.3510e-01  9.9060e-01  9.4790e-01  8.0670e-01  5.8220e-01  3.0080e-01
 -4.2000e-03 -3.0380e-01 -5.7610e-01 -8.0040e-01 -9.5150e-01 -1.0011e+00
 -9.3560e-01 -7.6930e-01 -5.4430e-01 -3.1980e-01 -1.4580e-01 -4.2300e-02]
```


Original signal
1
0
−1
0.0
0.2
0.4
0.6
0.8
1.0
Time (s)
Decimated signal
1
0
−1
0.0
0.2
0.4
0.6
0.8
1.0
Time (s)
Interpolated signal
1
0
−1
0.0
0.2
0.4
0.6
0.8
1.0
Time (s)

EXPERIMENT 13: Extraction and Plotting of Audio Frequency Components

Aim

To calculate the FFT of an audio signal, plot its frequency components and reconstruct it using IFFT.

MATLAB Code

```
clc; clear all; close all;

[s,fs] = audioread('ringtone.mp3');

N = 20;
s = s(2:1:2^N+1);

FFT_org_abs = fft(s);

subplot(3,1,1);
plot(s);
title('Original Audio Signal');

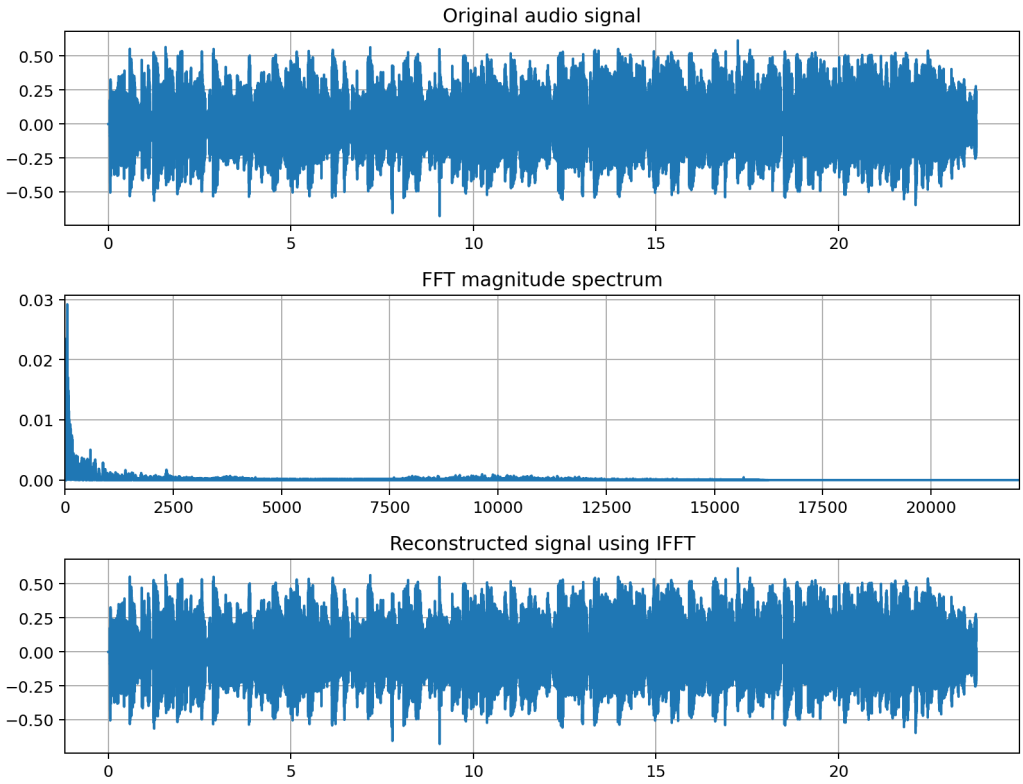
subplot(3,1,2);
stem(abs(FFT_org_abs));
title('Magnitude of FFT');

IFFT_org = ifft(FFT_org_abs);

subplot(3,1,3);
plot(IFFT_org);
title('Reconstructed Audio Signal');
```

Generated Output

```
Audio source: job_masuk (1).mp3
Sampling frequency: 44100 Hz
Samples analysed: 1048576
Duration: 23.7772 seconds
Maximum reconstruction error: 0.0000000000
```



EXPERIMENT 14: Filtering of Audio Frequency Components

Aim

To remove selected frequency components from an audio signal using frequency-domain filtering.

MATLAB Code

```
clc; clear all; close all;

[s,fs] = audioread('ringtone.mp3');

N = 20;
s = s(2:1:2^N+1);

FFT_org_abs = abs(fftshift(fft(s)));
nvec = -2^(N-1):2^(N-1)-1;

FFT_Filtered = fftshift(fft(s));

% Remove frequency components around zero
FFT_Filtered(abs(nvec) < 2^(N-6)) = 0;

% The original MATLAB indexing for the next
% range is invalid/reversed and needs correction.

filtered_signal = ifft(ifftshift(FFT_Filtered));

subplot(3,1,1);
plot(s);
title('Original Audio');

subplot(3,1,2);
plot(abs(FFT_Filtered));
title('Filtered Spectrum');

subplot(3,1,3);
plot(real(filtered_signal));
title('Filtered Audio');
```

Generated Output

```
Audio source: job_masuk (1).mp3
Sampling frequency: 44100 Hz
Samples analysed: 1048576
Filter: Frequency-domain high-pass filter
Cutoff frequency: 300 Hz
Filtered audio saved as filtered_ringtone.wav
```

